TOBB University of Economics and Technology

Department of Computer Engineering

Department of Artificial Intelligence Engineering

AI Health System for Doctors and Patients

Test Plan Report

Abdullah Çöplü

Abdurrahman Doğru

Ece Derya

Zeynep Ergül

Table of Content

1. Introduction **3**
2. Project Scope **3**
3. Features to Be Tested **4**
4. Testing Methodology **6**
4.1. Unit Testing 6
4.3. System Testing 8
4.4. Performance Testing 9
4.5. User Testing 9
5. Test Environment **10**
5.1. Hardware 10
5.2. Software 10
6. Test Cases **11**
7. Risks and Limitations **12**
7.1. AI Model Development Stage 12
7.2. External AI Service Dependencies 12
7.3. Performance and Scalability Constraints 12
7.4. Medical Decision-Support Limitation 13
7.5. Localization and Language Support 13
7.6. Dataset and Test Coverage Limitations 13
8. Conclusion **13**
9. Requirement Traceability Matrix **14**

1. Introduction

This test plan describes how the AI Health System for Doctors and Patients will be tested to verify that the implemented platform works correctly and meets the defined project requirements. It covers verification of core workflows such as authentication and role-based access control, medical image analysis, and symptom analysis with patient-safe outputs. Testing will be performed using unit, integration, system, performance, user and controlled failure-simulation tests. Test outcomes will be recorded to confirm correctness, safety, traceability, and performance compliance.

2. Project Scope

Project Title: AI Health System for Doctors and Patients

Brief Description:

This project is a web-based AI-assisted medical decision-support platform designed for both doctors and patients. Doctors can create clinical cases, upload medical images such as X-ray, CT, MRI, Ultrasound, and receive AI analysis results with confidence information to support clinical review. Patients can enter symptoms as free-text and receive simplified, patient-safe guidance generated by an LLM-based workflow that verifies user consent, applies safety moderation rules, and appends a disclaimer to every patient-facing output. The system is designed with privacy and security requirements in mind, including role-based access control and traceable logging for critical actions.

Main Functionalities:

- User registration and authentication (login/refresh/logout) with role-based access control (Doctor/Patient/Admin)
- Case management for doctors (create and view cases)
- Medical image upload and AI-driven image analysis (with confidence handling)
- AI-based medical image inference and result generation (including confidence and explainability support)
- Patient consent management for symptom analysis (view/accept enforcement)
- Patient symptom submission and safe symptom analysis output generation (moderation + disclaimer enforced)
- Feedback and traceability features (audit logging of critical actions and events, monitoring/health checks)
- Data security mechanisms (encryption utilities)
- Localization support for Turkish and English outputs

3. Features to Be Tested

Feature 1: User authentication and session management

User registration, secure password hashing, JWT (JSON Web Token) generation, and authentication enforcement on protected API endpoints will be tested. The system will be verified to reject unauthorized access attempts and handle invalid credentials securely.

Feature 2: Role-Based Access Control and Consent Verification

The system's authorization mechanisms will be tested to ensure that specific API endpoints correctly enforce role-based limits (e.g., Doctor vs. Patient). Additionally, the system will be strictly tested to block any symptom submission workflows if mandatory patient consent is not provided.

Feature 3: Patient Symptom Workflow and Chat Session Persistence

The conversational symptom analysis pipeline will be tested from end to end. This includes processing patient symptom submissions, rejecting empty or invalid text inputs, and successfully creating, storing, and retrieving continuous chat sessions (session history) for persistent patient-AI interactions.

Feature 4: Data Integrity, Validation, and Schema Enforcement

The application's data validation layer (Pydantic schemas) will be heavily tested. The system must be verified to catch and safely reject malformed requests, such as invalid email formats, missing required fields, or unsupported data types, ensuring backend stability.

Feature 5: AI-Based Medical Inference and Database Repositories

The core database repositories will be tested for reliable CRUD (Create, Read, Update, Delete) operations. The backend infrastructure integrating the PyTorch-based Deep Learning model (ResNet18) will also be verified for stable initialization and latency-compliant inference generation.

Feature 6: Doctor annotation, feedback, and override workflows

Doctor interactions with AI outputs will be tested, including adding annotations, submitting structured feedback, overriding AI-generated results, recording justification notes, and ensuring all such actions are permission-controlled and traceable.

Feature 7: Patient symptom submission and LLM-based symptom analysis

The patient symptom analysis workflow will be tested from end to end, including symptom text submission, language-aware prompt generation, LLM invocation, structured output parsing, persistence of symptom-related results, and creation of patient-safe analysis outputs.

Feature 8: Moderation and patient-safe output enforcement

Safety-critical output processing will be tested to verify moderation decisions, rewriting behavior, blocking behavior, removal of unsafe or anxiety-inducing medical language, and safe replacement of inappropriate, misleading, or confusing patient-facing outputs.

Feature 9: Mandatory disclaimer attachment and safe response rendering

All patient-facing outputs will be tested to ensure that clear and visible medical disclaimers are always appended and that no unsafe or raw model-generated response is shown directly without appropriate safety transformation.

Feature 10: Logging, monitoring, and traceability

The system's logging and monitoring capability will be tested for critical operations such as authentication events, image inference, moderation decisions, feedback submission, annotation, override actions, and user interactions. Completeness and traceability of recorded events will also be verified.

Feature 11: Localization and language negotiation

Localization functionality will be tested for both Turkish and English, including translation key resolution, missing-key detection, fallback behavior, language selection, and consistent localization across frontend and backend responses.

Feature 12: Client-side role-based views and API communication

Doctor and patient views will be tested to verify correct rendering of role-specific interfaces, proper use of authentication tokens, correct inclusion of language headers, safe error handling, and accurate parsing of backend responses through secure RESTful APIs.

Feature 13: Multi-modal diagnostic support

The system will be tested to verify combined processing of image-based analysis and symptom-based interpretation when both inputs are available.

Feature 14: Device, infrastructure, and software integration support

The system will be tested for secure upload from multiple device types and for integration with cloud/GPU-supported infrastructure, LLM services, and computer vision/deep learning libraries used in the project.

Feature 15: Performance and latency-critical workflows

Performance-sensitive functions will be tested, especially the symptom analysis pipeline, image inference pipeline, feedback persistence flow, moderated-content replacement flow, and concurrent user handling, to verify compliance with latency and capacity requirements under normal and stressed conditions.

Feature 16: Model and moderation quality requirements

The system will be tested to verify that model accuracy, false negative rate, false positive rate, and moderation detection accuracy satisfy the specified thresholds defined in the requirements.

4. Testing Methodology

The AI Health System for Doctors and Patients is tested using a multi-level methodology in order to verify functional correctness, safety, reliability, security, performance, and usability. Since the project includes both conventional software components and AI-supported decision workflows, testing is designed not only to validate normal system behavior but also to ensure controlled operation under failures, unsafe outputs, and edge conditions. The overall methodology consists of unit testing, integration testing, system testing, performance testing, and user-oriented testing.

4.1. Unit Testing

Unit testing is used to verify the correctness of individual classes, methods, and policy components in isolation. At this level, each module is tested independently with mocked dependencies so that its internal logic can be validated under deterministic conditions.

For the User Management subsystem, unit tests verify authentication logic, session creation and validation, invalid credential handling, logout behavior, and role-based access rules for doctor and patient users.

For the Image Processing subsystem, unit tests are applied to image validation logic, supported file format checks, metadata validation, secure upload handling, task-specific

model selection, inference invocation, and structured result generation. Where applicable, timeout and retry behavior are also tested as part of safe execution control.

For the Symptom Analysis subsystem, unit tests focus on symptom submission validation, LLM response handling, moderation decision processing, safe text rewriting, filtering of anxiety-inducing or overly technical medical language, disclaimer attachment, and structured patient-safe result generation.

For the Feedback and Moderation subsystem, unit tests verify doctor feedback submission, annotation creation, override operations, permission checks, moderation decision logic for AI-generated outputs, and safe replacement of blocked or moderated content.

For the Localization and UI support subsystem, unit tests validate language resolution, translation key lookup, fallback language behavior, and missing-key coverage checks for Turkish and English resources.

For the Monitoring and Logging subsystem, unit tests verify log creation, monitoring event generation, and traceability-related recording behavior for critical actions.

4.2. Integration Testing

Integration testing is used to verify interactions between multiple modules and to ensure that controller–service–repository flows behave correctly as a whole. Since the system follows a layered architecture, integration testing is essential for confirming that data, state transitions, and safety decisions propagate correctly across subsystem boundaries.

In the authentication flow, integration tests verify the interaction between controllers, authentication services, session-related components, and persistence layers. These tests confirm that valid access requests are processed correctly and unauthorized or invalid access attempts are handled safely.

In the doctor image analysis flow, integration testing validates the complete pipeline from image upload to secure storage, model inference, result persistence, and doctor-facing response generation. The interaction between image handling components, inference services, model selection components, and result repositories is tested together.

In the patient symptom analysis flow, integration tests verify the full path from symptom submission to LLM invocation, moderation, output sanitization, disclaimer attachment, and safe response delivery. These tests are particularly important because they validate that patient-facing outputs remain controlled and understandable.

In the feedback and override flow, integration tests verify that doctor actions are correctly passed through authorization checks, saved in the corresponding repositories, reflected in result-related records, and recorded in logs.

In the monitoring and traceability flow, integration tests validate that important actions such as login, inference execution, moderation decisions, feedback submission, and override operations produce correct trace records without disrupting primary business workflows.

In the localization flow, integration tests verify that frontend language selection, API request headers, backend language negotiation, and disclaimer localization remain consistent across the full request–response cycle.

Integration tests also verify multi-modal processing when both image and symptom inputs are available, as well as integration with LLM services and computer vision/deep learning libraries.

4.3. System Testing

System testing is used to validate the behavior of the complete AI Health System as an end-to-end product. At this level, the system is executed as a fully integrated environment and tested against real functional scenarios derived from the requirements and low-level design.

End-to-end system testing covers the main user journeys for the two primary user roles. For patients, the system is tested for login, symptom submission, receipt of low-risk suggestions, disclaimer visibility, localized response presentation, protection from anxiety-inducing terminology, and controlled handling of blocked or rewritten AI outputs. For doctors, the system is tested for login, image upload, AI result review, annotation creation, feedback submission, override actions, review of symptom-based and laboratory-related interpretations, and access to previous patient interactions.

System testing also includes negative and fail-safe scenarios. Examples include invalid inputs, unsupported file types, unauthorized access attempts, delayed responses,

moderation-triggered blocking, and service interruptions. In all such cases, the system is expected to return controlled responses and preserve traceability through recorded logs.

4.4. Performance Testing

Performance testing is used to evaluate the responsiveness, efficiency, and operational stability of the platform under realistic and stress conditions. This is especially important because the project contains latency-sensitive AI components and healthcare-related workflows where delayed responses may reduce usability and trust.

For the symptom analysis workflow, performance testing measures end-to-end response time from patient submission to safe output generation. The main objective is to verify that the LLM-supported pipeline, including moderation and disclaimer attachment, operates within the defined latency target under normal conditions.

For the medical image workflow, performance testing measures upload processing time, inference latency, and result delivery time under repeated and concurrent requests.

Performance testing also evaluates whether feedback entries are stored within the required time limit, whether moderated or blocked content is replaced within the specified response window, and whether the system can support at least 100 concurrent users with minimal performance degradation.

At this level, validation-oriented performance checks also cover model accuracy, false negative rate, false positive rate, and moderation detection performance against the specified thresholds.

4.5. User Testing

User testing is used to evaluate the usability, understandability, and practical correctness of the system from the perspective of its target users. Since the platform contains different interfaces for doctors and patients, user-oriented testing is necessary to confirm that the workflows are understandable and that the presented outputs are appropriate for each role.

For the patient interface, user testing focuses on the clarity of symptom submission screens, readability of low-risk AI-generated suggestions, visibility of disclaimers, understandability of non-technical explanations, and general ease of interaction in both

Turkish and English. Special attention is given to whether the output language is understandable and whether the patient can clearly distinguish supportive AI guidance from professional medical diagnosis.

For the doctor interface, user testing evaluates the usability of image upload, AI result review, symptom/laboratory interpretation review, annotation entry, feedback submission, override operations, and history access. The goal is to verify that doctors can efficiently inspect AI outputs and exercise human-in-the-loop control without ambiguity.

User testing may be carried out through representative scenarios, walkthrough-based evaluation, or sample-user trials. In addition to correctness, it helps identify interface-level issues such as poor navigation, unclear terminology, inadequate disclaimer visibility, or insufficient clarity in multilingual presentation.

5. Test Environment

5.1. Hardware

A laptop computer is used as the main testing machine for backend, frontend, and database execution.

GPU support is used for AI model inference and training-related tests. Standard network connectivity is required for external AI service communication. For image-related tests, sample medical image files are used.

5.2. Software

The software environment includes a standard operating system Windows; Python 3.12 for backend and AI modules; a REST-based backend service environment; a browser-based frontend for Doctor, Patient, and Admin views; a database system for persistence; localization resources for Turkish and English; encryption and anonymization utilities; audit logging and monitoring support; and testing tools for unit, integration, system, and performance testing. External AI dependencies such as LLM and moderation services are tested either through controlled integration endpoints or mocked environments.

6. Test Cases

Test ID	Description	Input(s)	Expected Output	Result
TC-01	Verify user creation and retrieval in DB	Valid User Model data	User record successfully created and fetched	**PASSED**
TC-02	Fetch user by email address from DB	Existing user's email string	Returns the correct User object	**PASSED**
TC-03	End-to-end patient symptom workflow	Patient Auth Token + Symptom details	Successful analysis and DB record creation	**PASSED**
TC-04	Enforce auth on symptom endpoints	Request without Bearer Token	*401/403* HTTP Status (Unauthorized/Forbidden)	**PASSED**
TC-05	Check consent requirement for symptoms	Valid Token, *Consent=False*, Symptom Text	*403* HTTP Status (Consent required)	**PASSED**
TC-06	Complete full symptom analysis flow	Valid Token, *Consent=True*, Symptom Text	*200 OK*, valid AI prediction and risk level	**PASSED**
TC-07	Verify password hashing mechanism	Raw string password	Hashed string (not matching raw input)	**PASSED**
TC-08	JWT access token generation	User credentials (ID, Role)	Valid, decodable JWT token string	**PASSED**
TC-09	Retrieve existing chat session	Existing session_id	Returns the correct chat session history	**PASSED**
TC-10	Create a new chat session automatically	No session_id + Initial message text	Generates and returns a new session_id	**PASSED**
TC-11	Validate user creation schema (Positive)	Valid email and password structure	Schema validation passes	**PASSED**
TC-12	Validate user creation schema (Negative)	Invalid email format (e.g., "test@com")	Schema validation throws ValidationError	**PASSED**
TC-13	Validate symptom submit request schema	Valid text string	Schema validation passes	**PASSED**
TC-14	Block empty symptom submit requests	Empty string ("")	Schema validation throws ValidationError	**PASSED**
TC-15	Verify role-based UI visibility for Doctor and Patient	Doctor login, Patient login	Correct role-specific screens/features are visible	**PASSED**
TC-16	Verify patient result screen shows disclaimer and non-technical safe output	Valid symptom submission	Valid symptom submission Disclaimer is visible; output is understandable and non-technical	**PASSED**
TC-17	Verify doctor can view previous patient/chat interaction history	Valid doctor session + existing history/session	Prior interactions are displayed correctly	**PASSED**

TC-18	Verify Turkish/English language presentation in UI	Switch language to TR and EN	User-facing texts change correctly	**ONGOING**
TC-19	Verify doctor feedback / annotation / override workflow	Valid doctor action payload	Feedback/annotation/override is saved successfully	**PASSED**
TC-20	Verify doctor medical image upload and AI result viewing	Valid doctor token + supported medical image	Upload succeeds and doctor can view AI result	**PASSED**
TC-21	Verify logging/monitoring record creation for critical actions	Trigger symptom analysis / doctor action	Corresponding log/monitoring record is created	**PASSED**
TC-22	Verify response-time and concurrency requirements	Timed requests / concurrent users	Meets defined latency and load expectations	**ONGOING**
TC-23	Verify model evaluation metrics for all models	Test dataset + trained model	Accuracy and related metrics are produced successfully	**ONGOING**
TC-24	Verify FNR and FPR calculation for critical classes	Validation predictions + labels	FNR and FPR values are computed correctly	**ONGOING**

7. Risks and Limitations

7.1. AI Model Development Stage

The AI models used for medical image analysis and symptom interpretation are still under training and optimization. While the system has already integrated the model inference pipeline and successfully produced valid outputs during testing (e.g., TC-06, TC-20), final model evaluation metrics are still being computed. Test cases related to model evaluation metrics and error analysis such as accuracy, false negative rate (FNR), and false positive rate (FPR) are currently ongoing (TC-23, TC-24). Therefore, the current testing verifies correct system integration and inference execution rather than the final predictive performance of the models.

7.2. External AI Service Dependencies

The symptom analysis pipeline relies on external AI services and moderation components. Although integration tests confirm that the workflow functions correctly and produces safe outputs (TC-03, TC-16), the system may still be affected by external service limitations such as API latency, temporary service unavailability, or rate limits. These factors may influence response time and reliability in real-world deployments.

7.3. Performance and Scalability Constraints

Performance and concurrency testing is currently ongoing (TC-22). The system architecture is designed to support multiple concurrent users and latency-sensitive AI workflows. However, most tests are currently executed in a limited development

environment using a single machine. As a result, real-world system performance may vary depending on infrastructure capacity, network conditions, and available computational resources.

7.4. Medical Decision-Support Limitation

The system is designed as a decision-support platform rather than a clinical diagnostic tool. Although safety mechanisms such as moderation filtering, mandatory disclaimers, and controlled patient-facing outputs are implemented and tested (TC-05, TC-16), AI-generated responses may still contain uncertainty. To mitigate this risk, doctor feedback, annotation, and override workflows are included in the system (TC-19), allowing human supervision over AI-generated results.

7.5. Localization and Language Support

The system supports both Turkish and English localization for user-facing interfaces and responses. However, full validation of language-switching behavior and multilingual consistency is still ongoing (TC-18).

7.6. Dataset and Test Coverage Limitations

Current testing relies on controlled datasets, predefined symptom inputs, and sample medical images. While this allows reliable validation of functional behavior, real-world clinical data may contain more diverse inputs, unexpected edge cases, and varying image quality. Additional testing with larger and more diverse datasets may be required to further improve robustness.

8. Conclusion

This test plan outlines the strategy used to verify the correctness, reliability, and safety of the AI Health System for Doctors and Patients. The testing process covers the main functional components of the system, including authentication and authorization mechanisms, patient symptom analysis workflows, AI-based medical image processing, moderation and safety controls, logging and monitoring mechanisms, and multilingual user interface behavior.

The implemented test cases demonstrate that the majority of the core system functionalities operate as expected. Critical workflows such as user authentication, role-based access control, symptom submission validation, safe output generation with disclaimers, doctor interaction workflows, and database operations have been successfully verified through unit, integration, and system tests. These tests confirm that the platform performs its core functions correctly and maintains secure and controlled behavior under both normal and invalid input conditions.

Some testing activities are still ongoing, particularly those related to performance evaluation and AI model quality metrics. Performance tests aim to confirm that the system can handle concurrent requests while maintaining acceptable latency levels. In addition, model evaluation tests are being conducted to measure accuracy and error-related metrics such as false negative rate (FNR) and false positive rate (FPR).

Overall, the testing performed so far confirms that the system architecture, data validation mechanisms, and AI inference integration operate reliably and safely. As the remaining performance and model evaluation tests are completed, the system will be further validated to ensure compliance with the defined requirements and quality standards. The results of these tests will also help guide future improvements and optimizations for the platform.

9. Requirement Traceability Matrix

Requirements

For clarity and traceability, closely related requirements have been grouped under broader requirement statements in this matrix. In particular, closely connected performance and quality criteria such as accuracy, false negative rate, false positive rate, and moderation performance are represented in a consolidated form, since these metrics are evaluated together during ongoing model validation activities. Similarly, highly related task-specific model requirements are presented in a generalized manner to keep the matrix concise while still preserving requirement coverage.

Requirement ID	Requirement Description
R-01	The system shall provide role-based functionality for doctors and patients through appropriate user interfaces.
R-02	The system shall allow doctors to upload medical images and review AI-assisted diagnostic results, symptom-based analyses, laboratory interpretations, and previous patient interactions.
R-03	The system shall allow patients to submit symptoms in natural language and receive simplified, non-technical, low-risk AI-supported outputs.
R-04	The system shall ensure that every patient-facing output includes a clear medical disclaimer and does not expose unnecessary anxiety-inducing medical terminology unless clinically required.
R-05	The system shall support secure diagnostic image upload from multiple devices and operate on cloud/GPU-supported infrastructure suitable for AI inference.

R-06	The system shall integrate with LLM-based symptom processing and computer vision / deep learning components for AI-supported health analysis.
R-07	The system shall expose secure RESTful APIs and implement logging/monitoring mechanisms for tracing user interactions, system events, and model decisions.
R-08	The system shall support doctor-side medical image analysis, doctor feedback/annotation/override actions, and patient-side symptom submission with non-critical health suggestions.
R-09	The system shall support safety-oriented AI behavior, including moderation/filtering of unsafe outputs, early warning assistance where applicable, and multi-modal analysis when both symptom and image inputs are available.
R-10	The system shall use task-specific AI models for image-based and NLP-based analysis workflows.
R-11	The system shall meet defined response-time and throughput expectations, including doctor image analysis latency, patient symptom analysis latency, feedback persistence time, moderated-content replacement time, and concurrent-user handling.
R-12	The system shall support Turkish and English localization with full user-facing translation coverage.
R-13	The system shall include a doctor feedback capability and retain feedback/moderation records to support traceability and system improvement.
R-14	The AI and moderation components shall satisfy target quality levels; evaluation of model accuracy, FNR, FPR, and moderation detection performance is ongoing and will continue during system development and validation.

Requirement Traceability Matrix

Requirement ID	Requirement Description	Related Test Case(s)
R-01	Role-based doctor and patient interfaces	**TC-04, TC-08, TC-15**
R-02	Doctor image upload and result review	**TC-17, TC-20**

R-03	Patient symptom input and safe AI output	**TC-03, TC-06, TC-13, TC-14, TC-16**
R-04	Disclaimer and anxiety-safe patient output	**TC-05, TC-06, TC-16**
R-05	Secure multi-device upload and AI infrastructure	**TC-20**
R-06	LLM and AI library integration	**TC-03, TC-06, TC-20**
R-07	Secure APIs with logging and monitoring	**TC-03, TC-04, TC-05, TC-06, TC-21**
R-08	Doctor AI actions and patient symptom support	**TC-03, TC-06, TC-19, TC-20**
R-09	Safe AI behavior, moderation, and multimodal support	**TC-05, TC-06, TC-16**
R-10	Task-specific AI model usage	**TC-06, TC-20**
R-11	Performance, latency, and concurrency requirements	**TC-22**
R-12	Turkish and English localization coverage	**TC-18**
R-13	Feedback capability and traceability retention	**TC-19, TC-21**
R-14	Ongoing AI and moderation quality validation	**TC-23, TC-24**